Ticket: HDFC Bank Campus Wallet

Customer: HDFC Bank campus wallet desk
Need: One **Account** blueprint that cannot be constructed. Savings and wallet **IS-A** account. Same loop prints both. Wallet can **pay** through a `Payable` contract.

Teaching-order: `ABC`, `@abstractmethod`, interface-style ABC (`Payable`). Validate with `print` + `return` — **no `raise` / `try` / `except`**. No type hints required.

Starter

Complete `kind()`, `add_interest`, and `WalletAccount.pay` in `starter/campus_wallet.py`. Keep balance on the abstract parent (`_balance` or `__balance`).

Broken

`Account` is concrete (`Account(...)` works). `kind()` always returns `"account"`. Wallet has a misspelled `paay` — not the `Payable.pay` contract. Checkout uses `if kind == ...`.

Done when

- [ ] `Account` inherits `ABC` — `Account("Campus", 0)` is commented / would TypeError
- [ ] `SavingsAccount` and `WalletAccount` implement `kind()`
- [ ] `WalletAccount` inherits `Payable` and `pay` uses `withdraw` (print + return)
- [ ] One loop prints both kinds without `if a.kind() == ...` for the slip
- [ ] Balance is not casually overwritten from outside
- [ ] No `raise` / `try` / `except`
- [ ] No type hints required

Stretch

- Tiny CLI menu (Deposit / Withdraw / Pay / Interest / List / Quit) around the list.
- `CurrentAccount` with a different withdraw rule (still print + return).
- **Later — Exceptions week:** `raise` on bad amounts; catch in the menu.